

Software Safety

- **Software safety** (sometimes called software system safety) is an engineering discipline that aims to ensure that software, which is used in [safety-related systems](#) (i.e. safety-related software), does not contribute to any hazards such a system might pose.
- There are numerous standards that govern the way how safety-related software should be developed and assured in various domains.
- Most of them classify software according to their criticality and propose techniques and measures that should be employed during the development and assurance:
 - Software for generic electronic safety-related systems: [IEC 61508^{\[1\]}](#) (part 3 of the standard)
 - Automotive software: [ISO 26262^{\[2\]}](#) (part 6 of the standard)
 - Railway software: EN 50716^[3]
 - Airborne software: [DO-178C/ED-12C^{\[4\]}](#)
 - Air traffic management software: DO-278A/ED-109A^[5]
 - Medical devices: [IEC 62304^{\[6\]}](#)
 - Nuclear power plants: IEC 60880^[7]

... Software Safety

- Software safety issues become important when computers are used to control real-time, safety-critical processes.

DESIGN FOR SAFETY

- System safety has an accepted order of precedence for applying safety design techniques.
- At the highest level, a system is intrinsically safe if it is incapable of generating or releasing sufficient energy or causing harmful exposures under normal or abnormal conditions (including outside forces and environmental failures) to create a hazardous occurrence, given the equipment and personnel in their most vulnerable condition [Malasky 19821].

DESIGN FOR SAFETY

- If an intrinsically safe design is not possible or practical, the preferred alternative is a design that prevents or minimizes the occurrence of hazards.
- This can be accomplished in hardware through such techniques as monitoring and automatic control (e.g., automatic pressure relief valves, speed governors, limit-level sensing controls), lockouts, lockins, and interlocks [Hammer 1972].
- A lockout device prevents an event from occurring or someone from entering a dangerous zone.
- A lockin is provided to maintain an event or condition.

DESIGN FOR SAFETY

- An interlock ensures that a sequence of operations occurs in the correct order; that is, it ensures that event A does not occur
 - (1) by accident (e.g., a preliminary, intentional action B is required before A can occur),
 - (2) while condition C exists (e.g., an access door is placed on high-voltage equipment so that when the door is opened, then the circuit is opened),
 - and (3) before event D (e.g., the tank will fill only if the vent valve has been opened first).

DESIGN FOR SAFETY

- The next, less desirable choice is a design that controls the hazard, if it occurs, using automatic safety devices.
- This includes detection of hazards and fail-safe designs, as well as damage control, containment, and isolation of hazards.
- The lowest precedence design option is to provide warning devices, procedures, and training to help personnel react to the hazard.

DESIGN FOR SAFETY

- Many of these system safety design principles are applicable to software.
- Note that software safety is not an afterthought to software design; it needs to be designed in from the beginning. There are two general design principles:
 - (1) the design should provide leverage for the certification effort by minimizing the amount of verification required and simplifying the certification procedure, and
 - (2) design features to increase safety must be carefully evaluated in terms of any complexity that might be added. An increase in complexity may have a negative effect on safety (as well as reliability).

DESIGN FOR SAFETY

- A safe software design includes not only standard software engineering and fault tolerance techniques to enhance reliability, but also special safety features.
- The emphasis here is on those design features directly related to safety.
- Risk can be reduced by reducing hazard likelihood or severity, or both.
- Two approaches are possible:
 - Hazards can be prevented,
 - or they can be detected and treated.
- Both have drawbacks; prevention of hazards tends to involve reducing functionality or inhibiting design freedom, whereas detection of hazards is difficult and unreliable.

Preventing Hazards through Software Design

- Preventing hazards through design involves designing the software so that faults and failures cannot cause hazards;
 - either the software design is made intrinsically safe
 - or the number of software hazards is minimized.
- Software can create hazards through omission (failing to do something required) or commission (doing something that should not be done or doing it at the wrong time or in the wrong sequence).
- Software is usually tested extensively to try to ensure that it does what it is specified to do. Owing to its complexity, however, it may be able to do a lot more than the software designers specified (or intended).
- **Design features can be used to limit the actions of the software.**

Preventing Hazards through Software Design

- As an example, it may be possible to use modularization and data access limitation to separate noncritical functions from critical functions and to ensure that failures of noncritical modules cannot put the system into a hazardous state; that is, they cannot block the operation of the safety-critical functions.
- The goal is to reduce the amount of software that affects safety (and thus reduce the verification effort involved) and to change as many potentially critical faults into noncritical faults as possible.

HUMAN FACTORS ISSUES

- As computers take over more and more monitoring and control functions in systems in which they are required to interact with humans, software engineers will need to consider human factors issues, especially with respect to software requirement specifications.
- Several issues arise with regard to safety.

HUMAN FACTORS ISSUES

- When designing a system that humans and computers will interact to control, one of the basic problems is the allocation of tasks between the human and computer.
- The goal is to optimize with respect to such criteria as speed of response, deviations of important variables, availability, and safety.
- It may not be possible to optimize all the variables because of conflicts, and therefore trade-offs must be considered.

HUMAN FACTORS ISSUES

- When designing a system that humans and computers will interact to control, one of the basic problems is the allocation of tasks between the human and computer.
- The goal is to optimize with respect to such criteria as speed of response, deviations of important variables, availability, and safety.
- It may not be possible to optimize all the variables because of conflicts, and therefore trade-offs must be considered.

HUMAN FACTORS ISSUES

- An essential ingredient in solving the task allocation problem is knowledge of the ways in which multiple tasks may interact and subsequently degrade or enhance the performance of the human or computer.
- Two or more tasks may be complementary in that having responsibility for all of them leads to improved performance on each because they provide important information about each other.
- On the other hand, tasks can be mutually incompatible in that having responsibility for all of them degrades performance on each [Rouse 1981]

HUMAN FACTORS ISSUES

- Rouse [1981] notes that there are two possible approaches to task allocation:
 - (1) partition the tasks into two subsets, giving one to the computer and one to the human,
 - or (2) dynamically allocate a particular task to the human or computer controller that has at the moment the most resources available for performing the task.
- The question of whether the best results are achieved by automating the human operator's tasks or by providing aids to help the operator perform has not yet been answered.

HUMAN FACTORS ISSUES

- The current trend is to have the human become more of a monitor and supervisor, and less of a continuous controller.
- As this happens, one of his or her primary responsibilities may be to detect system failures and diagnose their source.
- A particularly important issue in the area of failure detection concerns how the human's performance is affected by simultaneously having other tasks to do in addition to failure detection.

HUMAN FACTORS ISSUES

- Experimental data are conflicting [Ephrath and Young 1981; Rouse 1981; Shirley 1982].
- Rouse [1981] suggests that having to control while monitoring for failures is beneficial if performing the control task provides cues that directly help to detect failures and if the work load is low enough to allow the human to utilize the cues.
- Otherwise, controlling simply increases the work load and decreases the amount of attention that can be devoted to failure detection.

HUMAN FACTORS ISSUES

- Another aspect of complacency has been noted with regard to robots.
- For example, Park [1978] suggests that warning signals that a robot arm is moving should not be present continuously because humans quickly become insensitive to constant stimuli.
- Humans also commonly make mistaken assumptions about robot movements.
- For example, if the arm is not moving, they assume that it is not going to move; if the arm is repeating one pattern of motion, they assume that it will continue to repeat that pattern; if the arm is moving slowly, they assume that it will continue to move slowly; and if they tell the arm to move, they assume that it will move the way they want it to.

HUMAN FACTORS ISSUES

- There are other interesting issues with respect to safety and human factors.
- One is selecting the amount, type, and structure of information presented to the human under both normal and emergency conditions in order to optimize the human's performance.
- Another is maintaining human confidence in the automated system.
- For example, unless the pilot has confidence in an aircraft autoland system, he or she is likely to disconnect it instead of allowing the landing to be completed automatically [Mineck et al. 1972].

HUMAN FACTORS ISSUES

- A final issue is that of spurious shutdowns.
- Although it is important that the computer provide fail-safe facilities, evidence shows that, if the rate of spurious shutdowns or spurious warnings is too high, operators can be tempted to ignore them or to make unauthorized modifications in relevant devices to avoid them [Chamoux and Schmid 1983].

HUMAN FACTORS ISSUES

- Considering the much-repeated statement in this paper that mishaps often result from unanticipated events and conditions, it is doubtful that computers will be able to cope with emergencies as well as humans can.
- The emphasis should be on providing the human operator with an operational environment and appropriate information that will allow intervention in a timely and correct manner.
- Since this involves software requirements and design, it is important that
 - (1) software engineers become more familiar with human factors issues and
 - (2) requirement specification procedures and fault-tolerance techniques consider human/computer interaction.

Ethics in HCI

- Users are human.
- As HCI professionals we must be sure that our fellow humans perceive their encounter with usability and design professionals as pleasant without sacrificing the accuracy of our results.
- There are guidelines produced by professional organizations like the APA and the ACM about how HCI professionals should behave.
- However, there are few examples from real life about how to translate this information into everyday behavior.

Ethics in HCI - Ethics, HCI professional issues, social computing Informed consent and User Self-Esteem

- However, there are few examples from real life about how to translate this information into everyday behavior.
- Consider a simple example, the use of videotape in usability and field studies.
- Wendy Mackay, in one of the few papers specific to the HCI community [4], wrote of how easy it is to compromise our integrity when we employ video (or audio) to present our results and influence others.
- For example, a question that Mackay asks is “do we need to have internal participants who have already signed an employee agreement about being videotaped also sign a consent form?” Many HCI colleagues sacrifice the consent form for internal participants, but **this is probably an ethical violation since a video of an internal person doing poorly on a task could affect his/her reputation at work.**

Ethics in HCI - Ethics, HCI professional issues, social computing Informed consent and User Self-Esteem

- **Our belief is that internal participants should be afforded equal (or perhaps greater) protection than external participants, even to the extent of promising that their tape will not be shown to anyone other than the HCI team.**
- **A key ethical principle is that the participant should leave a test feeling no worse than when they arrived and ideally should feel better because they learned something, or contributed to a better design.**

Secrecy

- Field studies often present HCI colleagues with ethical dilemmas.
- For example, **a HCI colleague can sometimes find it difficult to preserve anonymity (and confidentiality) while at the same time providing feedback to customers on the results of field studies.**
- Customers often want to hear about the results of field studies and some may even demand to know as a condition for allowing interviews. How do you respond to a senior vice president who asks you “How did Fred do with the new product?” If your visit focuses on only a few users and you promised a report on your visit, how do you insure that the anonymity of the feedback is not compromised?
- There are several methods to insure anonymity:
 - put in writing the conditions of the visit and make sure that this is a prominent topic in preparations and planning.
 - You can also say that there will be an aggregate report from several different customers, but that no individuals will be identified.

Data Integrity and Presentation

- The manner in which we report the results of product evaluations presents some interesting ethical issues for HCI practitioners.
- Since we have to work with product teams, there may be a tendency to soft pedal bad results to preserve a working relationship with that team.
- The ability to simultaneously present bad results and keep good relationships is a difficult skill to learn but is important if we are to fulfill the user advocate role.
- Mackay [4] discusses how highlight tapes can be “treated” by omitting particular segments or by making “rare events appear representative”.
- **A final ethical issue in reporting is the validity and reliability of the data.**
At the 2000 UPA conference, a group of HCI practitioners discussed how few usability reports address validity and reliability, even at the most basic level. Are we being sloppy (and perhaps unethical) in presenting usability reports that fail to address fundamental aspects of data integrity?

Undue Persuasion (Urging)

- The design of persuasive technologies presents some ethical issues for HCI designers.
- [3] highlights ethical principles for the design of persuasive technologies.
- Here are two examples from their list:
 - Rule VII: “Persuasive technologies must not misinform in order to achieve their persuasive end.” ·
 - Rule VIII: “The creators of a persuasive technology should never seek to persuade a person or persons of something they themselves would not consent to be persuaded to do.”
- As HCI colleagues, what do we do when confronted with knowledge that a product may misinform potential users or persuade them to do something shameful? Do we remind our managers of these ethical issues, quit the company, or write an expose that may land us in court?

Life and Death

- **Ethical issues can become complex when people are working on systems that might have life or death consequences.**
- For example, designers of medical hardware and software make decisions that may compromise the health of an individual. Is it ethical to follow the advice that 5-8 users will reveal the major problems of a product?
- Testing 5-8 users is much cheaper than testing large numbers so there is a trade-off of time-to-market with possibility of death because a subtle problem did not come out in the limited testing.

Pushing Back on Management

- Pushing back on management when there are ethical issues (especially the less obvious ones) is hard for experienced practitioners and almost impossible for the brand-new person in the field.
- We view this panel as the starting point for a vigorous and continuing discussion about how we can keep high levels of integrity while working in product development environments that can bend ethical boundaries.

Examples

- HCI professionals face ethical issues every day. Here are some examples that could form the basis for a set of **case studies**.

Examples

- You go into the field. You have told your interviewees that their comments to you are private. A senior VP asks you “who did well in the study?” What do you say?
- Answer: **You can't give any answer that reveals or even narrows down the identity of an interviewee, no matter who is asking - a promise is a promise.**

Examples

- A usability test center entertains visitors by showing video clips with “funny” episodes from usability tests where test participants are picking their noses, etc.

Examples

- Your usability studies show that a product is not usable. Marketing folks claim that it is really usable. Do you email the CEO and lay your job on the line?
- Your usability studies show that a product is not usable. Marketing folks claim that it is really usable. Do you email the CEO and lay your job on the line?
- You are aware that a product you are working on is using some technology that someone has **patented** (there are lots of GUI patents now). Do you make a stink? Talk to your boss and ask him to talk to others?
- You learn that your company wants to acquire a company for some GUI technology, but you know from reading and reviewing an evaluation copy of the product that the technology is really pretty poor and even unreliable. Do you make a stink about this or let it pass until others discover they have wasted \$10,000,000 on clunky technology that they will write off in less than a year?

Examples - Conclusion

- As HCI professionals we cannot deny responsibility for the products of our work.
- We cannot defer responsibility to the decision makers, the managers and business sponsors that define the objectives of the system.
- We are responsible for ensuring that ethical issues in product development are discussed openly and not hidden away in bureaucratic closets.

References

- [LevesonSafety.pdf](#)
- DOI: 10.1145/634067.634197